Matheus dos Santos Lopes Rodrigues

Estimação adaptativa baseada em acurácia para tempo de execução de aplicações paralelas

Natal – RN

Janeiro de 2025

Matheus dos Santos Lopes Rodrigues

Estimação adaptativa baseada em acurácia para tempo de execução de aplicações paralelas

Trabalho de Conclusão de Curso de Engenharia de Computação da Universidade Federal do Rio Grande do Norte, apresentado como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação

Orientador: Samuel Xavier de Souza

Universidade Federal do Rio Grande do Norte – UFRN

Departamento de Engenharia de Computação e Automação – DCA

Curso de Engenharia de Computação

Natal – RN

Janeiro de 2025

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Rodrigues, Matheus Dos Santos Lopes.
Estimação adaptativa baseada em acurácia para tempo de execução de aplicações paralelas / Matheus Dos Santos Lopes Rodrigues. - 2025.
41f.: il.

Monografia (Graduação) - Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Engenharia de Computação, Natal, 2025.
Orientação: Prof. Dr. Samuel Xavier de Souza.

1. Análise de escalabilidade - Monografia. 2. Machine Learning - Monografia. 3. Aplicações paralelas - Monografia. I. Souza, Samuel Xavier de. II. Título.

RN/UF/BCZM CDU 004

Elaborado por Raimundo Muniz de Oliveira - CRB-15/429

Matheus dos Santos Lopes Rodrigues

Estimação adaptativa baseada em acurácia para tempo de execução de aplicações paralelas

Trabalho de Conclusão de Curso de Engenharia de Computação da Universidade Federal do Rio Grande do Norte, apresentado como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação

Orientador: Samuel Xavier de Souza

Trabalho aprovado. Natal – RN, 17 de Janeiro de 2025:

Prof. Dr. Samuel Xavier de Souza
UFRN

Prof. Dr. Ítalo Augusto Souza de Assis
UFERSA

Dr. Vitor Ramos Gomes da Silva
UFRN

Natal – RN
Janeiro de 2025

Aos meus pais.

AGRADECIMENTOS

Sou profundamente grato aos meus pais, Zenádio Lopes e Ângela dos Santos, pela educação e amor incondicional que sempre me ofereceram. Agradeço por todo o apoio, carinho e dedicação que me deram ao longo da vida.

Aos meus amigos — Reilta, Frankson, Júlia Costa, Maria Victória, Igor Dias, Kaique e Rafael Vinicius — estes que me acompanham desde o primeiro ciclo, agradeço por tornarem os dias mais proveitosos e leves com suas companhias. Vocês têm cadeira cativa em meu coração.

À minha namorada, Ana Beatriz, pelo seu amor e cuidado. Por estar constantemente ao meu lado, me oferecendo apoio e conselhos, e por me motivar a sempre buscar meu melhor.

Agradeço ao meu orientador e professor, Samuel Xavier de Souza, pelos valiosos ensinamentos, pelas orientações dedicadas, pela paciência e confiança depositada a este trabalho. Levarei em minha memória o seu discurso sobre a importância de aprender a aprender pro resto da minha vida.

Por fim, à UFRN por me proporcionar a educação necessária para me tornar quem sou hoje.

Being afraid isn't a bad thing.
It's my fear that brought me this far.
(Eisein, Sousou No Frieren)

RESUMO

A análise de escalabilidade permite a compreensão da eficiência com que um aplicativo paralelo utiliza os recursos de computação. Esse entendimento fornece informações sobre o comportamento do programa, permitindo otimizações, reduzindo o consumo de recursos e orientando as decisões de desenvolvimento para aplicativos de alto desempenho. O processo de análise consome muito tempo porque é necessário medir a execução do aplicativo em várias configurações. Estimar esses tempos de execução com precisão é um desafio devido às interações complexas entre o software e o hardware, o que pode levar a problemas como desequilíbrio de carga e contenção de recursos. Para lidar com esse problema, este documento propõe uma abordagem adaptativa usando a rede neural perceptron multicamada para prever o tempo de execução de algoritmos paralelos de forma iterativa, amostrando diferentes configurações de núcleos e tamanhos de entrada conforme necessário para garantir a precisão. O emprego da abordagem em três algoritmos paralelos diferentes (*Matrix Multiplication*, *Raytrace* e Busca em largura), em conjunto com três métodos diferentes de pré-processamento, validou a abordagem. A redução no tempo de execução da análise foi de até 88,2%, com um erro absoluto máximo de 7%. Integrado ao perfilador Pascal Analyzer, o modelo fornece previsões rápidas e precisas, otimizando o tempo de análise para pesquisadores e desenvolvedores. Ao acelerar a análise de escalabilidade e o cálculo de aceleração, o sistema torna o processo de avaliação do desempenho de aplicativos paralelos mais eficiente, contribuindo para avanços significativos na computação de alto desempenho.

Palavras-chaves: Análise de escalabilidade; Aprendizagem de máquina; Aplicações paralelas.

ABSTRACT

Scalability analysis allows the understanding of how efficiently a parallel application uses computing resources. This understanding provides insight into the behavior of the program, enabling optimizations, reducing resource consumption, and guiding development decisions for high-performance applications. The analysis process is very time-consuming because measuring application execution in several configurations is required. Estimating these execution times accurately is challenging due to the complex interactions between software and hardware, which can lead to problems such as load imbalance and resource contention. To deal with this problem, this paper proposes an adaptive approach using a multilayer perceptron neural network to predict the execution time of parallel algorithms iteratively by sampling different configurations of cores and input sizes as needed to ensure accuracy. Employing the approach on three different parallel algorithms (Matrix multiplication, Raytrace, and Breadth-first search), combined with three different pre-processing methods, validated the approach. The reduction in the time to perform the analysis was of up to 88.2%, with a maximum absolute error of 7%. Integrated with the Pascal Analyzer profiler, the model provides fast and accurate predictions, optimizing analysis time for researchers and developers. By accelerating scalability analysis and speedup calculation, the system makes the process of evaluating the performance of parallel applications more efficient, contributing to significant advances in high-performance computing.

Keywords: Scalability analysis; Machine Learning; Parallel applications.

LISTA DE ILUSTRAÇÕES

Figura 1 – Informações de Hardware do Supercomputador da UFRN 16
Figura 2 – Arquitetura de uma rede neural 18
Figura 3 – Arquitetura do *Pascal Analyzer* 20
Figura 4 – Fluxograma do *Pascal Regression Model* 25
Figura 5 – Arquitetura utilizada no modelo *Pascal Regression Model* 27
Figura 6 – Comparação do resíduo relativo absoluto entre a medição real e a estimativa do modelo *PascalRegressionModel* para diferentes estratégias de pré-processamento no algoritmo BFS. **(a)** Estratégia randômica. **(b)** Estratégia primeiro e último *core*. **(c)** Estratégia linear. 34
Figura 7 – Comparação do resíduo relativo absoluto entre a medição real e a estimativa do modelo *PascalRegressionModel* para diferentes estratégias de pré-processamento no algoritmo *Matrix Multiplication*. **(a)** Estratégia randômica. **(b)** Estratégia primeiro e último *core*. **(c)** Estratégia linear. 36
Figura 8 – Comparação do resíduo relativo absoluto entre a medição real e a estimativa do modelo *PascalRegressionModel* para diferentes estratégias de pré-processamento no algoritmo *Raytrace*. **(a)** Estratégia randômica. **(b)** Estratégia primeiro e último *core*. **(c)** Estratégia linear. 38
Figura 9 – Comparação do tempo total de execução entre a medição real e as estimativas do modelo para cada algoritmo e pré-processamento 39

LISTA DE ABREVIATURAS E SIGLAS

CPU	*Central Processing Unit*
HPC	*High Performance Computing*
IA	*Inteligência Artificial*
IMD	*Instituto Metrópole Digital*
JSON	*JavaScript Object Notation*
LAPPS	*Laboratório de Arquiteturas Paralelas para Processamento de Sinais*
ML	*Machine Learning*
MLP	*Multilayer Perceptron*
NPAD	*Núcleo de Processamento de Alto Desempenho*
UFRN	*Universidade Federal do Rio Grande do Norte*
Pthreads	*POSIX Threads*

SUMÁRIO

1 INTRODUÇÃO . . . 11
1.1 Objetivos . . . 11
1.2 Estrutura do Trabalho . . . 12

2 REFERENCIAL TEÓRICO . . . 13
2.1 Computação paralela . . . 13
2.2 Parâmetros de desempenho . . . 14
2.2.1 *Speedup* . . . 14
2.2.2 Eficiência . . . 14
2.2.3 Escalabilidade . . . 15
2.3 *Ambientes de High Performance Computing* . . . 15
2.4 *Machine Learning* . . . 16
2.5 Redes Neurais . . . 17
2.6 *Pascal Analyzer* . . . 19
2.6.1 Instrumentação e uso . . . 20

3 METODOLOGIA . . . 23
3.1 Pré-processamento . . . 23
3.2 *Pascal Regression Model* . . . 24
3.3 Arquitetura da MLP . . . 27

4 RESULTADOS E DISCUSSÕES . . . 28
4.1 Aplicativos de estudo de caso . . . 28
4.1.1 *Matrix Multiply* . . . 28
4.1.2 *Raytrace* . . . 29
4.1.3 *BFS* . . . 29
4.2 Arquitetura de estudo de caso . . . 30
4.3 Experimentos . . . 30
4.4 Resultados e Análises . . . 32

5 CONCLUSÃO . . . 40

REFERÊNCIAS . . . 41

1 INTRODUÇÃO

Nos anos recentes, *clusters* [1] de computação de alto desempenho têm aumentado o número de núcleos significativamente (ATTIG; GIBBON; LIPPERT, 2011). Como consequência, os usuários desses sistemas desejam tirar o máximo proveito desse grande número de núcleos, escalando suas aplicações (NISHTALA et al., 2009).

Devido à complexa interação entre as unidades de processamento em sistemas HPC, muitas aplicações podem sofrer ineficiências de desempenho quando escaladas para um grande número de núcleos. Problemas como comunicação e sincronização entre *threads*, desbalanceamento de carga e disputa por recursos compartilhados são frequentemente os principais obstáculos (PACHECO, 2011). Para garantir o uso eficiente do sistema, é crucial conhecer o comportamento da aplicação diante desses desafios antes de executá-la utilizando um número elevado de recursos.

Conforme mencionado, lidar com a complexidade inerente das aplicações paralelas torna a estimativa precisa do tempo de execução um desafio. Para superar essa dificuldade, a comunidade científica investiga o uso de *Machine Learning* (ML) para prever o desempenho dessas aplicações. Especificamente, alguns trabalhos no estudo de caso mostram que é possível prever com precisão o tempo de execução de aplicações na área de computação paralela (IPEK et al., 2005; CURTIS-MAURY et al., 2007; SINGH et al., 2007).

Desse modo, com a necessidade de acelerar a análise de escalabilidade [2] e *speedup*[3] em aplicações paralelas, este trabalho pretende construir um modelo de ML para estimar o tempo de execução total de programas paralelos em cenários de configurações distintas. Por meio da integração com o perfilador *Pascal Analyzer*, desenvolvido por pesquisadores do LAPPS da Universidade Federal do Rio Grande do Norte, o modelo otimizará o tempo dos pesquisadores, desenvolvedores e usuários em geral da ferramenta, fornecendo previsões rápidas e confiáveis.

1.1 Objetivos

1. Desenvolvimento de um modelo de ML capaz de estimar o tempo total de aplicações sob diferentes configurações de múcleos e complexidades de entrada;

[1] Arquitetura de sistema que une dois ou mais computadores como se fossem apenas um.

[2] Capacidade de um algoritmo ou programa paralelo manter um bom desempenho à medida que o número de processadores e o tamanho do problema envolvido na execução aumenta.

[3] Medição para a relação proporcional entre o tempo de execução sequencial e o tempo de execução em paralelo.

2. Integrar o modelo com a ferramenta *Pascal Analyzer*; e

3. Avaliar as estimativas do modelo em diferentes aplicações.

1.2 Estrutura do Trabalho

Este trabalho apresenta uma introdução sobre o tema, mostrando os fatores que motivam a implantação da ideia, além da justificativa e dos objetivos. Em sequência, o Capítulo 2 aborda todo o referencial teórico para a compreensão deste trabalho, desde os conceitos básicos sobre *machine learning* e redes neurais ao ramo do paralelismo, como HPC e as métricas de desempenho. O Capítulo 3, por sua vez, sendo o núcleo da contribuição, explica a metodologia empregada para a integração e treinamento do modelo na biblioteca *Pascal Analyzer*, enquanto o Capítulo 4 trata de apresentar a configuração para os experimentos, bem como as aplicações de caso de estudo e a análise dos resultados alcançados. O Capítulo 5 traz as principais conclusões e contribuições deste trabalho.

2 REFERENCIAL TEÓRICO

O objeto de estudo deste trabalho se fundamenta em conceitos e técnicas da inteligência artificial, como *machine learning* e redes neurais, e também do ramo do paralelismo, como HPC e as métricas de desempenho: escalabilidade, *speedup* e eficiência. Por fim, a estrutura da biblioteca *Pascal Analyzer* onde será realizada a integração do modelo. Nesta seção, então, será aprofundado o conhecimento sobre tais temas, elucidando sua relevância para o presente trabalho.

2.1 Computação paralela

Em 1965, Gordon Moore previu que o número de transistores em um chip dobraria a cada ano na década seguinte (MOORE, 1965). Essa afirmação, conhecida como Lei de Moore, levou o setor de processadores a dobrar o poder de processamento a cada um ou dois anos. No entanto, desde o início dos anos 2000, esse cenário começou a mudar. Acredita-se que os limites físicos dos processadores estão se aproximando, e a solução para aumentar ainda mais a capacidade de processamento dos computadores é a computação paralela (PACHECO, 2011).

A computação paralela refere-se à execução simultânea de tarefas computacionais em vários processadores com o objetivo de acelerar a solução de problemas complexos e com uso intensivo de recursos (FOSTER, 1995). Essa estratégia tem como objetivo ampliar a capacidade de processamento e lidar com problemas bastante complexos, frequentemente empregando dois métodos principais: o paralelismo de tarefas e o paralelismo de dados. No primeiro, diversas operações são alocadas entre os processadores para serem executadas de forma simultânea. No paralelismo de dados, por sua vez, o conjunto de dados é fragmentado entre os núcleos, permitindo que cada um realize a mesma operação ou uma operação similar em diferentes segmentos dos dados (PACHECO, 2011).

Para garantir uma implementação eficaz de programas paralelos, é fundamental entender os diferentes modelos de programação paralela, como a memória compartilhada e a memória distribuída. No modelo de memória compartilhada, os processadores têm acesso às mesmas áreas de memória, possibilitando que diversas *threads* executem simultaneamente partes do programa. Ferramentas como OpenMP, que simplifica a paralelização do código, e Pthreads, uma API padrão para o gerenciamento de *threads*, são bastante populares nessa abordagem. Em contrapartida, no modelo de memória distribuída, cada processador possui sua própria memória independente, e a comunicação entre eles ocorre por meio da troca de mensagens, utilizando a biblioteca MPI (*Message Passing Interface*).

É comum que as tarefas precisem cooperar umas com as outras. Essa cooperação geralmente envolve alguns núcleos que escrevem novos valores que outros precisam ler. Para que isso ocorra de forma eficiente, é essencial que os núcleos estejam sincronizados, com comunicação adequada e balanceamento de carga. Caso contrário, problemas como a "condição de corrida", que ocorre quando várias tarefas acessam os mesmos dados ao mesmo tempo, podem ocorrer, produzindo resultados inesperados (PATTERSON; HENNESSY, 2016).

Mesmo com o balanceamento de carga e a sincronização adequados, o aumento na velocidade do programa geralmente não é linear. A Lei de Amdahl afirma que a parte do programa que não pode ser paralelizada limita o ganho de velocidade da paralelização. Normalmente, qualquer problema consiste em partes que podem ser paralelizadas e partes que devem permanecer sequenciais (AMDAHL, 1967). Várias métricas são usadas para avaliar o comportamento de algoritmos paralelos, incluindo *speedup*, eficiência e escalabilidade (ABRANTES, 2008)

2.2 Parâmetros de desempenho

2.2.1 *Speedup*

Speedup é a proporção entre o tempo de execução do algoritmo sequencial e o tempo de execução do algoritmo paralelo (PACHECO, 2011). A fórmula de medição de *speedup* é dada por:

$$S(p) = \frac{T_s}{T_p} \tag{2.1}$$

Onde:

- T_s é o tempo de execução com um único processador.
- T_p é o tempo de execução com p processadores.

2.2.2 Eficiência

A eficiência é uma métrica que quantifica o quanto um algoritmo aproveita os recursos de processamento paralelo em comparação com sua execução sequencial. Ela é definida como a relação entre o aumento de velocidade e o número de processadores usados (PACHECO, 2011). A medida de eficiência é dada por:

$$E(p) = \frac{S(p)}{p} = \frac{T_s}{p \cdot T_p} \tag{2.2}$$

Onde:

- $S(p)$ é o *speedup* obtido com p processadores.
- p é o número de processadores usados.

2.2.3 Escalabilidade

Escalabilidade refere-se à capacidade de um programa de manter sua eficiência à medida que os recursos de computação (como o número de processadores) aumentam. Um programa é considerado escalável se manter sua eficiência com a adição de mais processadores. Um programa escalável pode ser classificado como fortemente escalável, se a eficiência não diminuir, fixando o tamanho da entrada e aumentando a quantidade de recursos computacionais, ou fracamente escalável, se a eficiência não diminuir com o aumento do tamanho da entrada e da quantidade de recursos computacionais na mesma proporção (PACHECO, 2011).

2.3 *Ambientes de High Performance Computing*

Ambientes de computação de alto desempenho, ou *High Performance Computing*, trata-se da prática de agrupar recursos computacionais de forma a alcançar um maior desempenho do que se teria com apenas um servidor ou computador (STERGIOU et al., 2018). Essa agregação de recursos computacionais pode exercer em forma de Supercomputadores personalizados ou *desktops* individuais, os chamados *clusters* (DONGARRA et al., 2019). A HPC pode ser executada localmente, na nuvem ou de forma híbrida, juntando ambas metodologias (AKIOKA; MURAOKA, 2010).

Em um *cluster*, cada computador é denominado nó e cada um deles realiza uma função distinta. Os nós podem operar tanto de maneira independente quanto de forma paralela, sendo esta última a mais significativa, pois permite a divisão de grandes problemas em subproblemas menores que são processados ao mesmo tempo (BARNEY, 2021). Essa habilidade de processamento paralelo, juntamente com uma comunicação eficaz entre os nós, possibilita que os *clusters* solucionem desafios complexos em tempos consideravelmente reduzidos se comparados àqueles que seriam viáveis em um único computador (BUYYA et al., 2009).

Os clusters tipicamente funcionam em sistemas de memória distribuída. Embora este trabalho aborde conceitos relevantes para ambientes de memória distribuída, seu foco principal recai sobre os sistemas de memória compartilhada.

Um exemplo de ambiente de computação de alto desempenho é o supercomputador da Universidade Federal do Rio Grande do Norte (UFRN), localizado no Instituto Metrópole

Digital (IMD) e gerenciado pelo Núcleo de Processamento de Alto Desempenho (NPAD). Este centro se destaca como uma referência no Nordeste em ciência de alto desempenho e, atualmente, o NPAD presta serviços a mais de 500 pesquisadores, operando com mais de 70 nós computacionais (conforme ilustrado na Figura 1). Esses nós permitem a geração rápida de dados e um eficiente processamento de informações para fins científicos (GOIS, 2024).

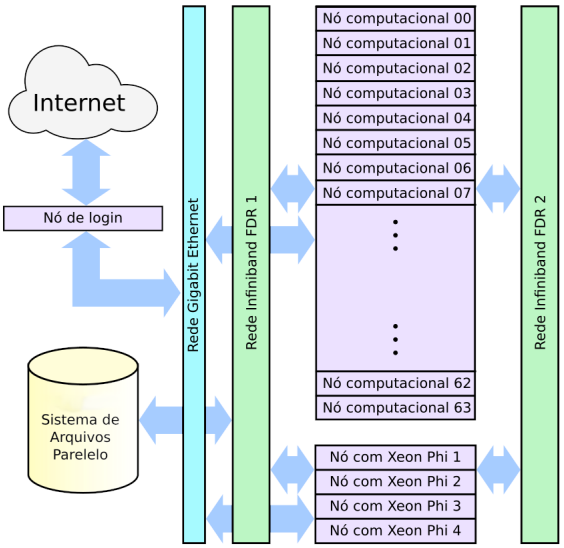

Figura 1 – Informações de Hardware do Supercomputador da UFRN

Fonte: http://npad.ufrn.br/npad/hardware

2.4 *Machine Learning*

Ao longo da história, a humanidade tem-se esforçado para desvendar as complexidades da mente e do universo, tentando compreender como uma pequena quantidade de matéria pode levar à percepção, manipulação do ambiente e habilidade de antecipação. A inteligência artificial (IA) eleva essa questão a um novo patamar, visando à criação de entidades inteligentes — sistemas artificiais com a capacidade de pensar, aprender e agir de maneira independente (RUSSEL; NORVIG, 2010).

A IA é uma área multidisciplinar que emergiu após o término da Segunda Guerra Mundial, mais precisamente em 1956, graças ao trabalho do professor John McCarthy. Este assunto abrange uma ampla variedade de subáreas, incluindo aprendizagem de máquina, percepção computacional, processamento de linguagem natural e diagnósticos médicos. A IA é reconhecida como uma disciplina universal pela sua adaptabilidade e habilidade de solucionar problemas em variados contextos intelectuais (RUSSEL; NORVIG, 2010).

Dentro da vasta gama de subáreas da inteligência artificial, uma das mais proeminentes e transformadoras é a aprendizagem de máquina. A aprendizagem de máquina

é um método de análise de dados que automatiza a construção de modelos analíticos, permitindo que os sistemas aprendam e se adaptem com base em dados (MITCHELL, 1997). Essa capacidade de aprender com os dados é o que diferencia a aprendizagem de máquina dos métodos tradicionais de IA.

Segundo Alpaydin (2014), a aprendizagem de máquina consiste em programar computadores para otimizar parâmetros de um modelo utilizando dados de treinamento ou experiências anteriores, podendo este ser preditivo, descritivo, ou ambos, para fazer previsões ou obter conhecimento a partir dos dados. A aprendizagem de máquina utiliza a teoria da estatística para criar modelos matemáticos e requer a ciência da computação para desenvolver algoritmos eficientes tanto para o treinamento e otimização dos modelos quanto para a representação e inferência após o aprendizado.

Existem três principais tipos de aprendizagem: supervisionada, não supervisionada e por reforço. Na aprendizagem supervisionada, os modelos são treinados usando dados rotulados, ou seja, dados que já possuem uma resposta conhecida. Na aprendizagem não supervisionada, os modelos são treinados usando dados sem rótulos, identificando padrões e estruturas nos dados. A aprendizagem por reforço envolve treinar um agente para tomar decisões sequenciais, recebendo recompensas ou punições com base nas ações tomadas. Nesse trabalho, focaremos na aprendizagem supervisionada, onde definimos o escopo e a técnica utilizada na criação do modelo deste trabalho.

2.5 Redes Neurais

Uma rede neural é um conjunto interconectado de elementos, unidades ou nós de processamento simples, cuja funcionalidade é vagamente baseada no neurônio animal. A capacidade de processamento da rede é armazenada nos pontos fortes ou pesos da conexão entre as unidades, obtidos por um processo de adaptação ou aprendizado a partir de um conjunto de padrões de treinamento (GURNEY, 1997).

Em outras palavras, uma rede neural (ou rede neural artificial) consiste numa camada de entrada de neurônios (ou nós, unidades), com n camadas intermediárias escondidas, seguida por uma camada final de neurônios de saída. A Figura 2 representa uma arquitetura típica, na qual as linhas que conectam os neurônios também são ilustradas. Cada ligação/conexão está associada a um número chamado peso. A saída y_i, do neurônio i na camada oculta é,

$$y_i = \text{ReLU}\left(\sum_{j=1}^{N} W_{ij} x_j + b_i\right), \tag{2.3}$$

onde ReLU() é chamado de função de ativação (ou função de transferência), N o

número de neurônios de entrada, W_{ij} os pesos, x_i entradas dos neurônios de entrada, e b_i bias (ou termo de bias) cuja finalidade é permitir que a função de ativação seja ajustada para torná-la mais flexível. O propósito da função de ativação é, além de introduzir um comportamento de não-linearidade na rede neural, limitar o valor do neurônio para que o modelo não seja prejudicado por neurônios divergentes. A escolha desse tipo de modelo de IA para este trabalho foi motivada justamente pela capacidade de representar a não-linearidade, permitindo capturar o comportamento das aplicações paralelas de forma mais eficaz. Um exemplo comum de função de ativação é a função ReLU definida na Equação 2.4,

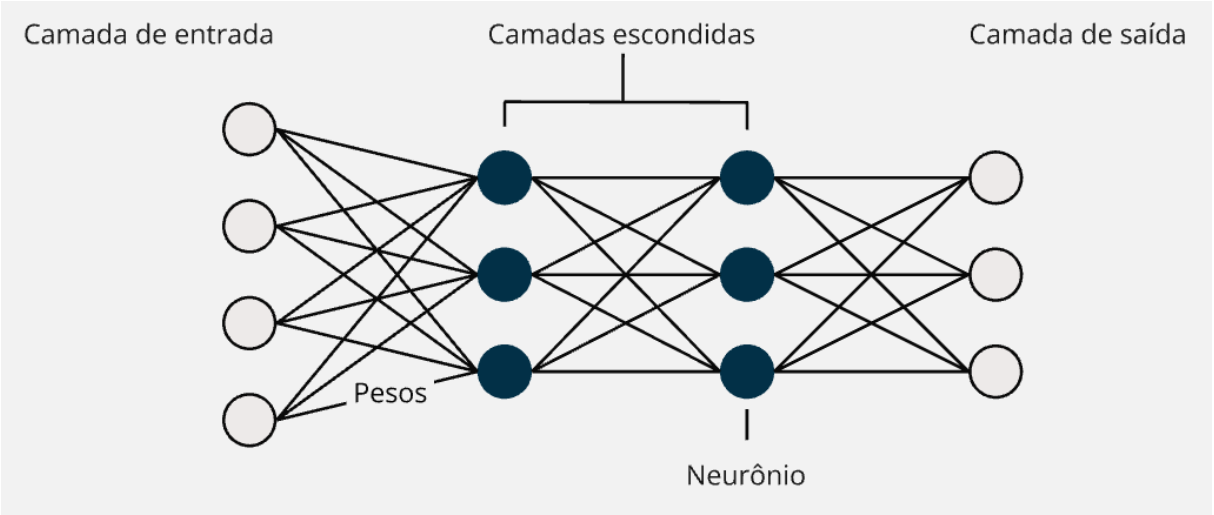

Figura 2 – Arquitetura de uma rede neural

Fonte: Autoria própria

$$\mathrm{ReLU}(x) = \max(0, x). \tag{2.4}$$

Existem outros exemplos de funções de ativação como arco tangente, arco hiperbólico e sigmoidal. Elas divergem na resposta das entradas em comparação a ReLU, entretanto possuem maiores intervalos de saída (WANG, 2003).

Dessa forma, foi demonstrado que uma rede neural construída da maneira acima pode aproximar qualquer função computável com uma precisão arbitrária. Os números fornecidos aos neurônios de entrada são variáveis independentes e os retornados pelos neurônios de saída são variáveis dependentes da função que está sendo aproximada pela rede neural. As entradas e saídas de uma rede neural podem ser binárias (como sim ou não) ou até mesmo símbolos (verde, vermelho, etc.) quando os dados são codificados adequadamente. Esse recurso confere uma ampla gama de aplicabilidade às redes neurais.

2.6 *Pascal Analyzer*

Projetado para avaliar e comparar várias execuções de aplicativos paralelos em várias configurações, o *Pascal Analyzer* oferece percepções significativas sobre o desempenho e a escalabilidade dos aplicativos paralelos. Ao superar as restrições das ferramentas convencionais, ele combina instrumentação automática, mapeamento direto de seções paralelas e redução de dados sem comprometer a precisão, permitindo uma análise completa com uma sobrecarga mínima no desempenho (menos de 1%). Isso facilita a identificação de gargalos e padrões de desempenho, promovendo a compreensão e a otimização de aplicativos de alto desempenho em diferentes contextos. (SILVA et al., 2022).

Para medir, combinar e comparar diferentes execuções de um aplicativo paralelo, a ferramenta baseia-se em dois conceitos essenciais: atuadores e sensores. Os atuadores correspondem a elementos externos do programa, como núcleos/*threads*, entradas do programa e a frequência de operação da CPU. Os sensores, por sua vez, atuam como observadores e medidores de alterações nos atuadores, possibilitando calcular e quantificar variáveis como tempo de execução, consumo de energia e contadores de desempenho (SILVA et al., 2022).

Com uma arquitetura modular, os elementos da ferramenta são mostrados na Figura 3. Ela mostra os atuadores e sensores que se conectam à parte central da biblioteca. Em seguida, há o *Pascal Analyzer*, que está representada por *wrapper library*; ele recebe código binário da própria biblioteca ou código-fonte que foi previamente instrumentado usando o módulo de instrumentação. Por fim, os dados coletados são salvos em um arquivo no formato JSON, que permite análise e visualização futuras em uma interface gráfica (GUI) (SILVA et al., 2022).

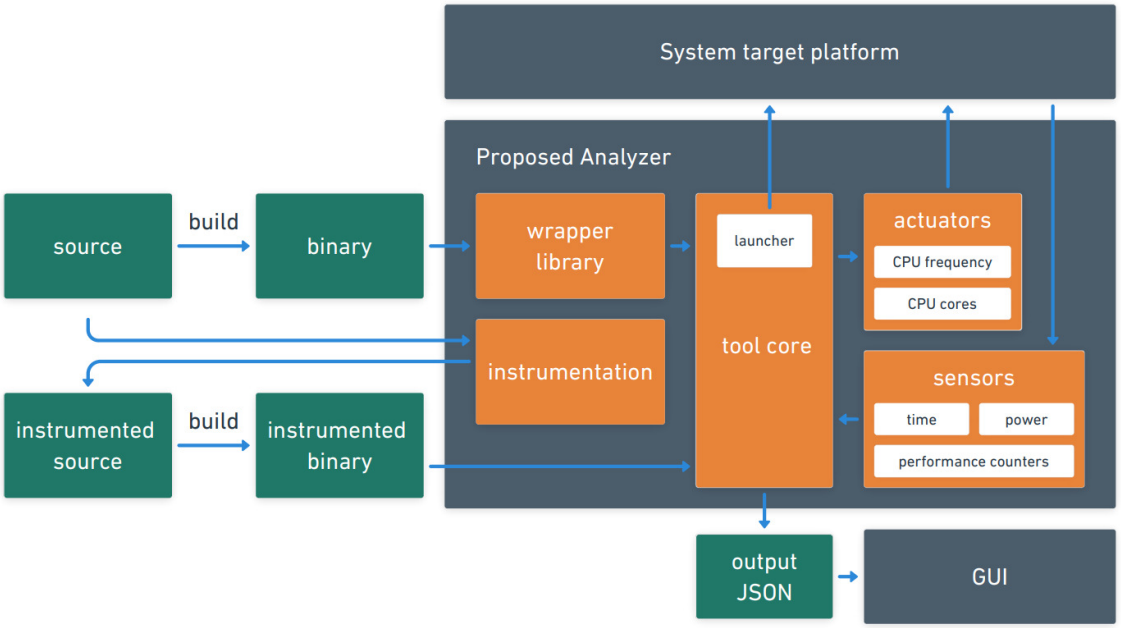

Figura 3 – Arquitetura do *Pascal Analyzer*

Fonte: (SILVA et al., 2022)

2.6.1 Instrumentação e uso

O módulo de instrumentação desempenha um papel fundamental no processo. Além de automatizar a instrumentação, ele também avalia a sobrecarga e o grau de intrusividade. Projetado com ênfase na eficiência e otimização, opera com um número reduzido de instruções. No momento, o módulo é compatível com a instrumentação de linguagens C/C++ por meio de bibliotecas compartilhadas, possibilitando tanto métodos automáticos quanto manuais. A instrumentação manual, em especial, é valiosa quando se precisa investigar partes específicas do código do programa, independentemente do que elas contêm (SILVA et al., 2022).

A instrumentação automática conta com uma rotina que intercepta a geração de *threads* por meio da variável de ambiente `LD_PRELOAD`. Essa rotina altera segmentos de uma biblioteca nativa já presente, possibilitando a interceptação de funções que gerenciam a criação de *threads*, como `pthread_create` na biblioteca pthread, `GOMP_parallel` (versão do GCC) e `__kmpc_fork_call` (versão do Clang) dentro do framework OpenMP, além de outras funções semelhantes em diversos compiladores. Assim, regiões paralelas são identificadas de forma automatizada (SILVA et al., 2022).

Um exemplo de instrumentação manual pode ser visto na Listagem 2.1. Nela é possível observar duas rotinas principais: `pascal_start` e `pascal_stop`. A primeira determina o início da região de interesse e, a segunda, o seu fim, capturando o tempo exato no momento da sua chamada. Ambas acompanham um parâmetro "1"que evidencia o *ID* da região de análise, de forma que sua utilização possa ser replicável para outros trechos

do código. Dessa forma, a instrumentação manual permite uma análise mais detalhada e personalizável.

Listing 2.1 – Exemplo de instrumentação manual com *Pascal Analyzer*

```
1  void BFS(vector<vector<int>> &graph, int start)
2  {
3      vector<bool> visited(graph.size(), false);
4      queue<int> q;
5
6      visited[start] = true;
7      q.push(start);
8
9      while (!q.empty())
10     {
11         int current = q.front();
12         q.pop();
13
14         #pragma omp parallel for
15         for (int i = 0; i < graph[current].size(); ++i)
16         {
17             pascal_start(1) // Inicio da regiao 1
18             int adjacent = graph[current][i];
19             {
20                 visited[adjacent] = true;
21                 q.push(adjacent);
22             }
23             pascal_stop(1) // Fim da regiao 1
24         }
25     }
26 }
```

Após a conclusão do perfilamento, os dados coletados são exportados para um arquivo .JSON e salvos no armazenamento de disco. Na Listagem 2.2, é demonstrado um exemplo do arquivo gerado pela biblioteca. Nele, é possível visualizar uma estrutura de dados contendo informações essenciais acerca do sistema e do comando usado no experimento, seguido pela descrição dos dados coletados, a lista de atuadores, bem como a representação das informações dos sensores. Por fim, as amostras são separadas por configurações de atuador e sensor.

Listing 2.2 – Exemplo de arquivo de saída gerado pelo *Pascal Analyzer*

```
1  {
2    "config": {
3    ########## Informacoes sobre o sistema, comando utilizado, etc. ##########
4      "pkg": "./bfs",
5      "execdate": "08-10-2024_17:24:51",
6      "command": "/home/mdslrodrigues/.conda/envs/tcc-pascal/bin/
           pascalanalyzer ./bfs -c 1:32 -i 1000000,1100000,...,2600000 -r 10 -o
           bfs_real_output_sp.json",
7      "hostname": "r1i3n14.npad.internal",
8      "kernel": "Linux-4.18.0-477.10.1.el8_8.x86_64-x86_64-with-glibc2.28",
9      "disable_hyperthread": false,
10     "auto_instrumentation": false,
```

```
11      "idle_time": 0,
12      ########## Descricao dos atuadores e sensores aplicados ##########
13      "data_descriptor": {
14        "values": ["start_time", "stop_time"],
15        "keys": ["cores", "input", "repetitions"]
16      },
17      "disable_cores": false,
18      "discard_ompnt": true,
19      ########## Entradas da aplicacao ##########
20      "arguments": [
21        "1000000",
22        "1100000",
23        "...",
24        "2600000",
25      ]
26    },
27    ########## Dados coletados ##########
28    "data": {
29          "32;0;1": {
30            "start_time": 1728419091.857216,
31            "stop_time": 1728419109.7968302
32          },
33          "32;0;2": {
34            "start_time": 1728419109.7975526,
35            "stop_time": 1728419127.6700299
36          },
37          {...},
38          "1;16;10": {
39            "start_time": 1728504049.011704,
40            "stop_time": 1728504051.5568795
41          }
42      }
43  }
```

3 METODOLOGIA

Nessa seção, é apresentada em detalhes a metodologia utilizada no *Pascal Regression Model*, incluindo as técnicas estatísticas aplicadas para avaliar os resultados no *Pascal Analyzer*. Além disso, também são abordadas as metodologias empregadas para as técnicas de pré-processamento dos dados.

3.1 Pré-processamento

O pré-processamento de dados é uma etapa crucial para garantir a qualidade e a integridade das informações utilizadas no treinamento do modelo. Neste trabalho, foram realizados os seguintes procedimentos de pré-processamento:

- **Normalização**: Cada coluna de entrada foi padronizada para valores dentro do intervalo [0, 1], garantindo uniformidade nas escalas dos dados.
- **Organização randômica do *dataset***: Os dados foram apresentados de forma aleatória, sem critérios específicos, com a intenção de incorporar estocasticidade à organização.
- **Organização do *dataset* por único núcleo e último núcleo**: Para estabelecer uma estrutura mais determinística e balanceada, os dados foram primeiramente organizados com base nas configurações de um único núcleo, seguidas pelas configurações com a última quantidade de núcleos definido pelo o usuário. As demais configurações ficaram dispostas de maneira aleatória.
- **Organização linear do *dataset***: As configurações foram classificadas de acordo com a soma das *features core* e *input*, em ordem crescente. Por exemplo, ao considerar as tuplas (1,0) e (5,3), onde o primeiro valor representa o *core* e o segundo o *input*, a configuração (1,0) vem antes de (5,3), visto que a soma resultante 1 é menor que 8. O objetivo dessa ordenação era, além de inserir um fator determinístico, reproduzir um comportamento semelhante ao da análise de escalabilidade fraca.

A razão para empregar essas três maneiras diferentes de organizar os dados é descobrir qual estratégia oferece o melhor desempenho ao modelo. Isso possibilita um uso mais eficaz dos dados e, por consequência, uma diminuição no erro de estimativa.

3.2 *Pascal Regression Model*

Para acelerar a avaliação da escalabilidade e do *speedup* em aplicativos paralelos — processos que podem ser lentos devido à complexidade do aplicativo ou à intensa interação entre os atuadores mencionados no subcapítulo anterior —, a proposta sugere a integração de uma *Multilayer Perceptron* (MLP) ao *Pascal Analyzer*. Essa abordagem baseia-se nas medições já realizadas, considerando as diferentes configurações de entrada e núcleos do programa, para estimar o tempo de execução em cenários futuros. Dessa forma, é possível inferir o desempenho do aplicativo em várias situações sem a necessidade de medir todas as configurações possíveis, o que reduz a sobrecarga de tempo e energia.

A metodologia apresentada adota uma abordagem cíclica para treinar e avaliar um modelo de inferência de tempo de execução em sistemas paralelos. Cada fase desse processo é descrita abaixo, conforme ilustrado no fluxograma da Figura 4. Considere sempre o *start* como a entrada de uma nova medição previamente realizada.

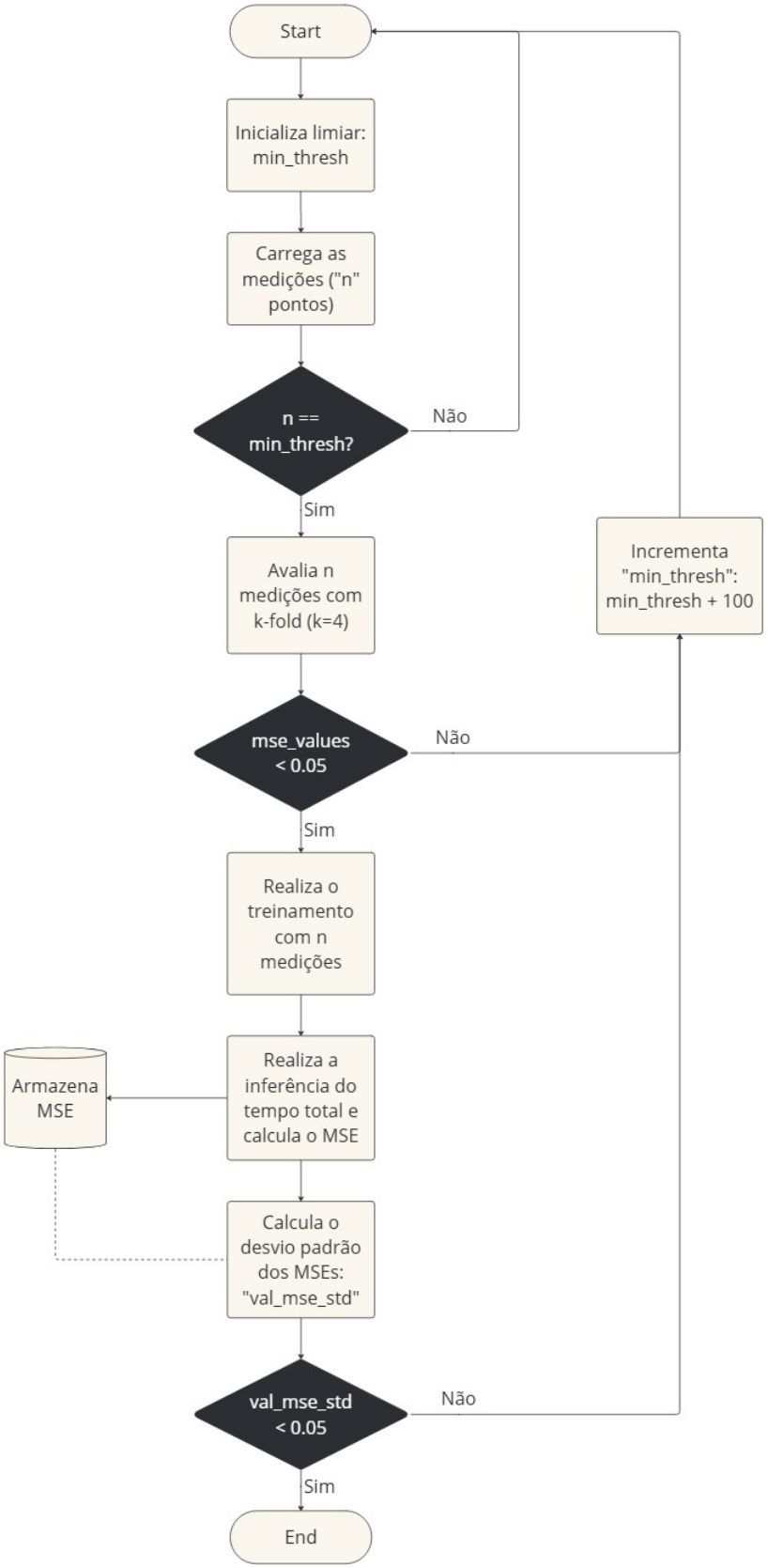

Figura 4 – Fluxograma do *Pascal Regression Model*

Fonte: Autoria própria

1. **Inicialização**: O processo começa com a especificação de um limite mínimo (`min_thresh`). Esse limite indica o número mínimo de medições de entrada necessárias para iniciar o treinamento do modelo.

2. **Carregamento das medições**: Em seguida, são carregadas as medições de entrada (n pontos), que representam configurações específicas da medição, como o número de núcleos e os parâmetros de entrada, usando dados coletados em execuções anteriores.

3. **Checagem na quantidade de medições**: É feita uma verificação para garantir que o número de medições n atenda ao valor mínimo, chamado `min_thresh`. Se n estiver diferente desse limite, o processo retorna à fase de carregamento de medições. Esse ciclo é repetido até que haja um número adequado de medições para a análise inicial.

4. **Avaliação com validação cruzada (k-fold)**: Uma vez atingido o número mínimo de medições necessárias, é realizada uma avaliação inicial usando a validação cruzada k-fold com k=4. Essa fase é essencial, pois permite verificar a adequação das medições e possibilitar o treinamento do modelo em sua totalidade posteriormente. Na avaliação, o erro quadrático médio (MSE) é calculado e o modelo só prossegue para a fase de treinamento se o `mse_value`, correspondente à média dos erros dos treinamentos dos 4 subconjuntos, for menor que 0,05. Vale ressaltar que, nesta etapa de validação cruzada, o modelo é avaliado de forma independente e os pesos gerados não são armazenados, servindo apenas para análise da adequação inicial. Essa aplicação da validação cruzada é fundamental para minimizar a sobrecarga: ao avaliar preliminarmente o desempenho com o critério de aceitação de MSE menor que 0.05, evitamos os custos computacionais do treinamento do modelo quando as medições não são de qualidade suficiente para garantir uma previsão precisa.

5. **Treinamento do modelo**: Se o valor de MSE estiver dentro dos padrões aceitáveis, o modelo será treinado com as medições n disponíveis.

6. **Inferência e cálculo final do MSE**: Após o treinamento, o modelo realiza uma inferência dos dados de treinamento, calculando o MSE final para as estimativas.

7. **Cálculo do desvio padrão dos erros**: O desvio padrão dos valores de MSE (`val_mse_std`) é calculado, permitindo assim que a consistência das previsões do modelo seja avaliada e fornecendo um mecanismo de parada caso ocorra convergência.

8. **Critério de convergência**: Caso o desvio padrão dos erros (`val_mse_std`) permaneça abaixo de 0,05, o procedimento é finalizado e o MSE final é anotado para referência posterior. Se isso não ocorrer, o `min_thresh` é incrementado novamente e o ciclo reinicia, assegurando uma adaptação gradual até que o modelo alcance os critérios de erro e consistência desejados.

Todos os códigos desenvolvidos ao longo deste projeto, incluindo o modelo descrito neste fluxograma, foram integrados à ferramenta *PascalSuite Analyzer*. O código completo está disponível no repositório da ferramenta no *GitLab*, na *branch* "pascalregressionmodel"(LAPPS, 2025).

3.3 Arquitetura da MLP

A arquitetura utilizada no modelo *Pascal Regression Model* pode ser visualizada na Figura 5. Ela acompanha uma camada de entrada com dois nós, seguidos de duas camadas escondidas, cada uma com 64 nós. Por fim, uma camada de saída, referindo-se ao tempo de execução total. Nos hiperparâmetros, foi adotada uma taxa de aprendizado de 0.001, com 1000 épocas por treinamento. Essa escolha dos hiperparâmetros foi realizada com base no método empírico, resultando em desempenhos mais estáveis e com menores resíduos.

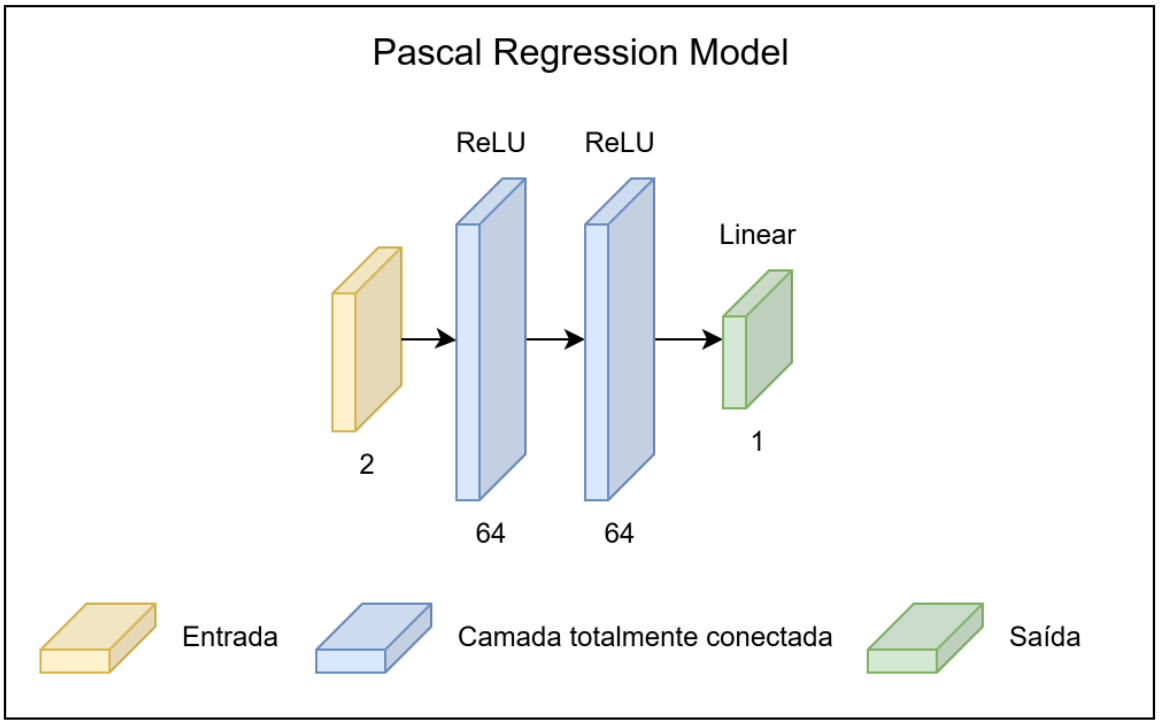

Figura 5 – Arquitetura utilizada no modelo *Pascal Regression Model*

Fonte: Autoria própria

4 RESULTADOS E DISCUSSÕES

Nesta seção, serão discutidas as medições e os resultados da estimativa para três situações diferentes com três pré-processamentos diferentes. Além disso, será apresentada a aplicação do modelo integrado à ferramenta de *profiling Pascal Analyzer*, detalhando a configuração dos experimentos, o que permitirá futuras comparações com outras plataformas de computação e a configuração de experimentos, entre outros aspectos.

4.1 Aplicativos de estudo de caso

O modelo *Pascal Regression Model* foi testado usando três algoritmos diferentes: *Matrix Multiply*, *Raytrace* e *Breadth-First Search* (BFS). Todos os algoritmos foram paralelizados usando diretivas OpenMP.

4.1.1 *Matrix Multiply*

O programa *Matrix Multiply* (Listagem 4.1) recebe duas matrizes quadradas e cria uma terceira matriz com o resultado da multiplicação. A diretiva `#pragma omp parallel for collapse(2)` é usada para paralelizar os loops que percorrem as linhas e colunas da matriz resultante, permitindo que várias iterações sejam executadas em diferentes *threads* simultaneamente. Essa cláusula instrui o compilador a tratar os loops aninhados como um único bloco, maximizando a eficiência da paralelização. O produto é calculado usando três laços aninhados: os dois primeiros iteram sobre a matriz de resultados, enquanto o terceiro soma os produtos dos elementos correspondentes das matrizes de entrada.

Listing 4.1 – Trecho do código do *Matrix Multiply* analisado

```
1  void matrix_multiply(
2      const vector<vector<int>>& A,
3      const vector<vector<int>>& B,
4      vector<vector<int>>& result
5  ) {
6      int n = A.size();
7      int m = B[0].size();
8      int p = B.size();
9
10     #pragma omp parallel for collapse(2)
11     for (int i = 0; i < n; i++) {
12         for (int j = 0; j < m; j++) {
13             result[i][j] = 0;
14             for (int k = 0; k < p; k++) {
15                 result[i][j] += A[i][k] * B[k][j];
16             }
17         }
```

```
    }
}
```

4.1.2 *Raytrace*

O *Raytrace* simula a interação de raios com esferas para determinar a cor de cada pixel em uma imagem 3D, atuando como um tracejador de luz simplificado. O uso do OpenMP paraleliza a renderização, distribuindo a carga de trabalho em vários threads para aumentar a eficiência. A imagem gerada contém esferas com cores fixas e um plano de fundo específico. A diretiva `#pragma omp parallel for collapse(2) schedule(dynamic)` paraleliza a geração de cada pixel na imagem. Cada raio é emitido do ponto de vista da câmera e projetado em uma grade 3D, permitindo que a cor correspondente a cada pixel seja calculada. O trecho de código paralelizado pode ser visualizado na Listagem 4.2 abaixo.

Listing 4.2 – Trecho do código do *Raytrace* analisado

```
    ...
    #pragma omp parallel for collapse(2) schedule(dynamic)
        for (int j = 0; j < height; ++j) {
            for (int i = 0; i < width; ++i) {
                float u = float(i) / width;
                float v = float(j) / height;
                Ray ray(Vec3(0, 0, 0), Vec3(2 * u - 1, 2 * v - 1, -1));
                image[j * width + i] = trace(ray, spheres);
            }
        }
    }
    ...
```

4.1.3 *BFS*

O algoritmo BFS (Listagem 4.3) implementa a exploração de grafos representados por listas de adjacência. A função usa um vetor visitado para marcar os vértices visitados e uma fila `q` para armazenar os vértices a serem explorados. O loop processa os vértices na fila até que ela esteja vazia e, para cada vértice atual, os vértices adjacentes são explorados. A diretiva `#pragma omp parallel for` é usada para paralelizar essa exploração, permitindo que vários vértices adjacentes sejam examinados simultaneamente, otimizando a execução do algoritmo.

Listing 4.3 – Trecho do código do BFS analisado

```
void BFS(vector<vector<int>> &graph, int start)
{
    vector<bool> visited(graph.size(), false);

    queue<int> q;

```

```
7       visited[start] = true;
8       q.push(start);
9
10      while (!q.empty())
11      {
12          int current = q.front();
13          q.pop();
14
15          #pragma omp parallel for
16          for (int i = 0; i < graph[current].size(); ++i)
17          {
18              int adjacent = graph[current][i];
19              visited[adjacent] = true;
20              q.push(adjacent);
21          }
22      }
23  }
```

4.2 Arquitetura de estudo de caso

Os experimentos foram conduzidos no supercomputador do NPAD/UFRN, mais precisamente em nós da partição intel-512. Estes nós são equipados com dois processadores Intel Xeon E5-2698 v3, cada um contando com dezesseis núcleos e um cache de 40 MB, funcionando a uma frequência máxima de 2,3 GHz, sem a ativação do turbo. Cada nó da partição dispõe de 512 GB de memória RAM. O sistema operacional em uso foi o Linux, com o *kernel* 4.18.

4.3 Experimentos

Nas Listagens 4.4, 4.5 e 4.6 estão os parâmetros de configuração para cada execução de cada estudo de caso: *Matrix Multiplication*, *Raytrace* e BFS, respectivamente.

A Listagem 4.4, por exemplo, ilustra um arquivo de submissão configurado para o gerenciador de workload SLURM (*Simple Linux Utility for Resource Management*). O SLURM é uma ferramenta amplamente utilizada em clusters de computação de alto desempenho (*High-Performance Computing* - HPC), permitindo o gerenciamento eficiente de recursos, como CPUs, memória e tempo de execução. Ele facilita a alocação precisa dos recursos necessários para as aplicações submetidas, otimizando o uso do cluster.

No script, cada parâmetro com o prefixo `#SBATCH` desempenha um papel fundamental na configuração do trabalho submetido:

- `-job-name`: Define um nome para o trabalho, facilitando sua identificação no sistema (neste caso, `matrix_multiplication`).

- `-partition`: Determina a partição do cluster a ser utilizada, indicando o conjunto de nós disponíveis para a tarefa (exemplo: `intel-512`).
- `-time`: Estabelece o tempo máximo permitido para a execução do trabalho (neste caso, 2 dias).
- `-cpus-per-task`: Especifica o número de CPUs a serem alocadas para a execução da tarefa (32 neste exemplo).
- `-hint`: Configura otimizações para o tipo de workload, como `compute_bound`, que é ideal para aplicações intensivas em processamento.
- `-exclusive`: Garante que o nó seja reservado exclusivamente para esta tarefa, evitando interferências de outras tarefas que poderiam impactar o desempenho.

Além das opções específicas do SLURM, o script também inclui variáveis de ambiente e comandos relacionados à aplicação. Por exemplo, a variável `OMP_NUM_THREADS` define o número de *threads* utilizadas pela aplicação paralela.

Essas configurações foram cuidadosamente elaboradas para garantir uma execução eficiente, evitando enviesamentos causados pela disputa de recursos computacionais nos nós alocados por outros usuários e/ou processos, além de assegurar a precisão do perfilamento realizado com o *Pascal Analyzer*.

Listing 4.4 – Script de submissão no SLURM para a execução da aplicação *Matrix Multiplication*, incluindo configurações dos parâmetros de perfilamento na ferramenta *Pascal Analyzer*

```
1  #!/bin/bash
2  #SBATCH --job-name=matrix_multiplication
3  #SBATCH --partition=intel-512
4  #SBATCH --time=2-0:0
5  #SBATCH --cpus-per-task=32
6  #SBATCH --hint=compute_bound
7  #SBATCH --exclusive
8
9  export OMP_NUM_THREADS=32
10
11 pascalanalyzer
12 ./matrix_multiplication     # binario da aplicacao
13 -c 1:32                     # threads/cores
14 -r 10                       # numero de repeticoes
15 --ipts 100,200,...,1400     # tamanho das matrizes quadradas
16 --model 1000 300            # epocas e min_threshold
17 -o mm_model_random_output_sp.json # arquivo de saida
```

Listing 4.5 – Linha de comando mostrando a configuração dos parâmetros de perfilamento da aplicação *Raytrace* via bash com o *Pascal Analyzer*

```
1  #!/bin/bash
```

```
2  #SBATCH --job-name=raytrace
3  #SBATCH --partition=intel-512
4  #SBATCH --time=2-0:0
5  #SBATCH --cpus-per-task=32
6  #SBATCH --hint=compute_bound
7  #SBATCH --exclusive
8
9  export OMP_NUM_THREADS=32
10
11 pascalanalyzer
12 ./raytrace  # binario da aplicacao
13 -c 1:32     # threads/cores
14 -r 10       # numero de repeticoes
15 --ipts 320 240,640 360,640 480,800 600,960 540,1024 768,1280 720,1280
       1024,1600 900,1920 1080,2160 1440,2560 1440,3200 1800,3840 2140,5120
       2880,7680 4320   # resolucoes de renderizacao
16 --model 1000 300    # epocas e min_threshold
17 -o raytrace_model_random_output_sp.json # arquivo de saida
```

Listing 4.6 – Linha de comando mostrando a configuração dos parâmetros de perfilamento da aplicação BFS via bash com o *Pascal Analyzer*

```
1  #!/bin/bash
2  #SBATCH --job-name=bfs
3  #SBATCH --partition=intel-512
4  #SBATCH --time=2-0:0
5  #SBATCH --cpus-per-task=32
6  #SBATCH --hint=compute_bound
7  #SBATCH --exclusive
8
9  export OMP_NUM_THREADS=32
10
11 pascalanalyzer
12 ./bfs           # binario da aplicacao
13 -c 1:32         # threads/cores
14 -r 10           # numero de repeticoes
15 --ipts 1000000,1100000,...,2600000  # numero de vertices
16 --model 1000 300                    # epocas e min_threshold
17 -o bfs_model_random_output.json     # arquivo de saida
```

Ao fim da execução, o comando produz um arquivo .json que contém todas as informações essenciais para nossa análise. Um exemplo de saída pode ser visto na Listagem 2.2. Com esse arquivo, podemos criar gráficos para observar, analisar a escalabilidade e avaliar as estimativas do modelo.

4.4 Resultados e Análises

Para avaliar o modelo, foram realizados testes com os três algoritmos sem aplicar o modelo e, em seguida, com o *Pascal Regression Model* sobre três metodologias de pré-processamento diferentes. O erro relativo absoluto foi usado como critério para analisar as estimativas de cada cenário de configuração dos atuadores (*cores* e *inputs*).

O modelo randômico, na Figura 6a, mostra uma maior quantidade de resíduos, principalmente em situações com um número reduzido de núcleos e em entradas finais. Essa característica pode ser atribuída à aleatoriedade introduzida na ordenação dos dados, que pode tornar mais difícil o reconhecimento de padrões complexos no tempo de execução. Por não seguir uma ordem determinística, o modelo enfrenta dificuldades para capturar adequadamente as tendências não-lineares associadas ao escalonamento da eficiência. Contudo, é possível notar que, em certas áreas de maior quantidade de núcleos e complexidade de entrada, o erro é baixo, sugerindo que o modelo pode oferecer estimativas satisfatórias em contextos específicos.

Por outro lado, o modelo que se fundamenta na utilização do primeiro e último *core* apresenta uma estrutura balanceada e determinística, resultando em uma distribuição mais uniforme dos resíduos. Como demonstrado na Figura 6b, os resíduos são menores em diversas regiões intermediárias, o que sugere que essa abordagem é mais eficaz para retratar o comportamento geral do tempo de execução. Contudo, resíduos mais elevados ainda se manifestam em algumas configurações intermediárias, indicando que o modelo enfrenta dificuldades em captar transições suaves associadas ao comportamento não-linear.

O modelo linear, que visa reproduzir um comportamento mais determinístico, que remete à escalabilidade fraca, revela-se ineficaz no contexto das aplicações paralelas estudadas. A Figura 6c mostra a presença de resíduos persistentes e significativos, especialmente nas configurações que utilizam um maior número de núcleos, onde o tempo de execução real tende a alcançar um ponto de saturação devido aos *overheads* e às partes seriais, em conformidade com a Lei de Amdahl. Mesmo assim, o modelo linear demonstra baixos erros em certas regiões, indicando que ele pode ser eficaz para configurações mais simples ou em situações em que a ligação entre *input*, núcleos e tempo de execução se assemelha a um comportamento linear.

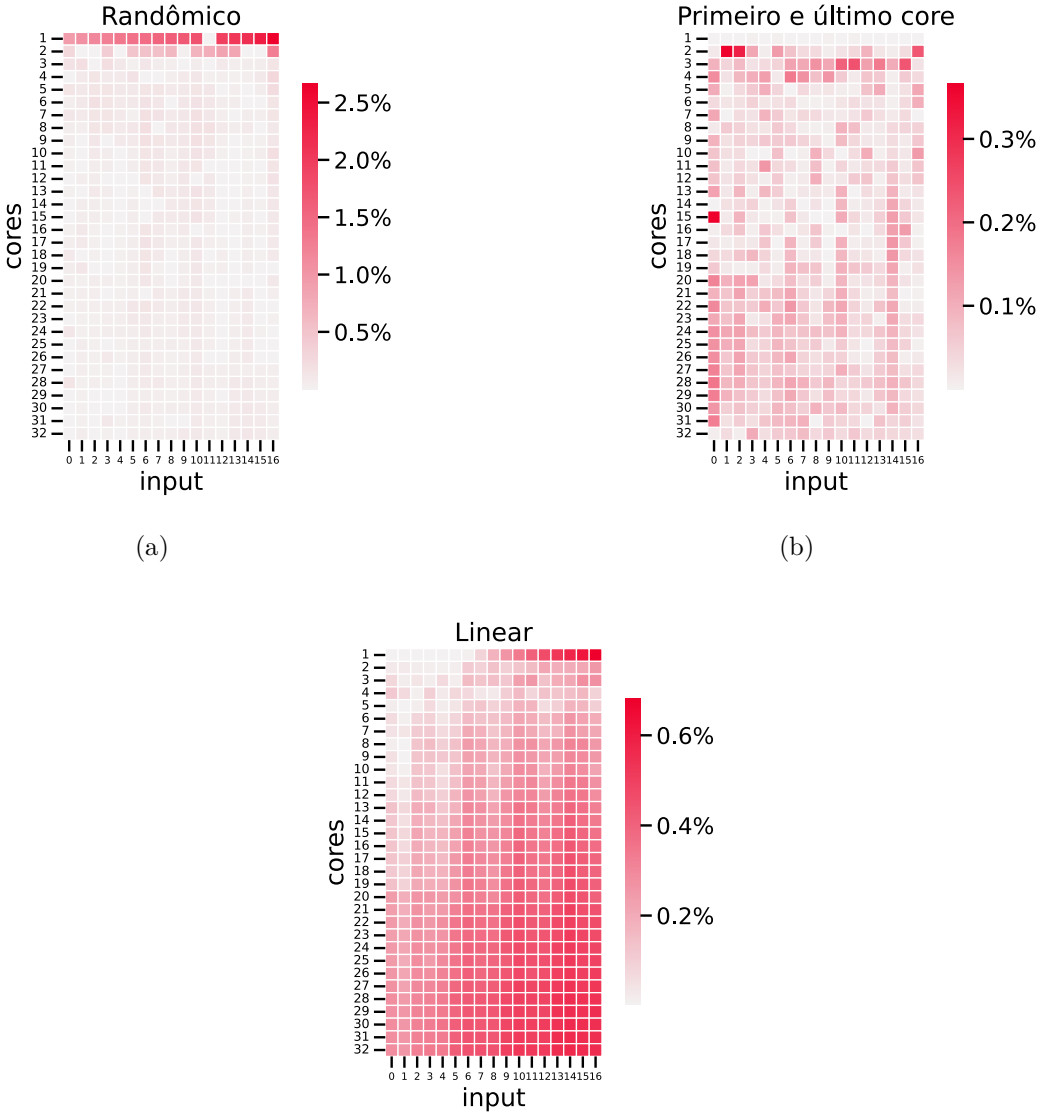

Figura 6 – Comparação do resíduo relativo absoluto entre a medição real e a estimativa do modelo *PascalRegressionModel* para diferentes estratégias de pré-processamento no algoritmo BFS. **(a)** Estratégia randômica. **(b)** Estratégia primeiro e último *core*. **(c)** Estratégia linear.

Na avaliação dos modelos utilizando o algoritmo *Matrix Multiplication*, conforme apresentado na Figura 7, nota-se uma diferença marcante em relação ao cenário anterior.

O modelo randômico, na Figura 7a, demonstra resíduos bastante elevados, especialmente em situações que envolvem um número reduzido de *cores* e *inputs* iniciais. A magnitude dos resíduos, que ultrapassa 20%, indica a dificuldade do modelo em compreender o comportamento da aplicação nessas condições. Essa situação pode ser atribuída à organização estocástica dos dados durante o pré-processamento, resultando na não visitação ou sorteio dessas configurações durante a fase de treinamento. Esses casos geram

muitos *outliers* devido à limitação de recursos computacionais, o que impede a identificação de padrões consistentes. Por outro lado, para configurações que utilizam um maior número de núcleos e entradas, observa-se uma redução dos resíduos, o que sugere que o modelo consegue se adaptar e capturar comportamentos onde os efeitos do paralelismo são mais fortes.

O modelo que utiliza o primeiro e o último *core*, demonstrado na Figura 7b, novamente apresenta um desempenho superior em relação aos outros. A organização dos dados de forma determinística permite ao modelo capturar com mais precisão a escala e os padrões do tempo de execução. Os resíduos são significativamente menores, com a maioria dos valores abaixo de 4%, e a distribuição é mais equilibrada em todas as combinações de *cores* e *inputs*. Observa-se que os maiores resíduos ocorrem em configurações intermediárias de paralelismo, o que indica que o modelo ainda enfrenta desafios para lidar com os efeitos não lineares que aparecem durante o processo de paralelização, como os *overheads* de sincronização ou comunicação. Contudo, de maneira geral, esse modelo demonstra ser robusto e eficaz na estimativa do tempo de execução sob diferentes configurações.

O modelo linear, por sua vez, na Figura 7c, revela erros residuais altos em áreas onde há um incremento no número de núcleos, porém, desta vez, isso acontece juntamente com entradas menores. Esse fenômeno pode ser explicado pela natureza do *overhead* de comunicação, que se torna mais evidente nessas situações. Com o aumento no número de núcleos, o custo de sincronização e a troca de informações entre as unidades de processamento se elevam, o que torna o tempo de execução menos previsível e dificulta a habilidade do modelo linear de prever com exatidão esses padrões. Apesar disso, em configurações com entradas maiores, o modelo apresenta um desempenho relativamente melhor, pois o impacto dos *overheads* se dilui com o aumento da carga computacional.

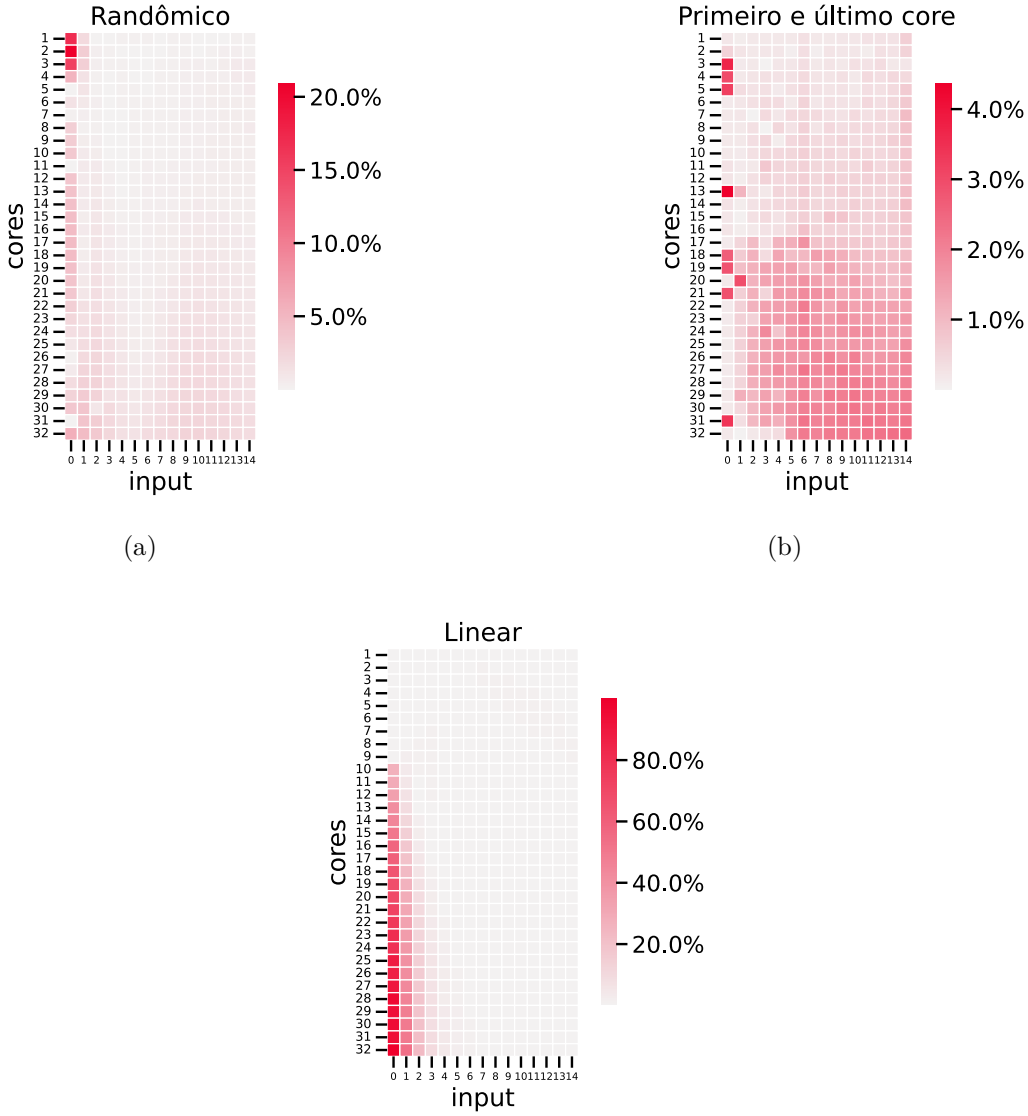

Figura 7 – Comparação do resíduo relativo absoluto entre a medição real e a estimativa do modelo *PascalRegressionModel* para diferentes estratégias de pré-processamento no algoritmo *Matrix Multiplication*. **(a)** Estratégia randômica. **(b)** Estratégia primeiro e último *core*. **(c)** Estratégia linear.

Nos testes realizados com o algoritmo *Raytrace*, na Figura 8, os padrões identificados nos resíduos absolutos entre as medições reais e as previsões do modelo são similares aos comportamentos observados em experimentos que envolvem os outros algoritmos, principalmente ao *BFS*, porém com magnitude de erros menores — menor que 11%.

No modelo aleatório, exibido na Figura 8a, nota-se novamente uma dispersão maior de erros, especialmente em configurações que utilizam um número reduzido de núcleos e entradas pequenas. Essa tendência destaca a dificuldade do modelo em generalizar, resultado da aleatoriedade na organização dos dados, um padrão que se repete em outros

experimentos. A variabilidade dos dados nessas áreas pode gerar valores extremos que impactam negativamente a capacidade preditiva do modelo.

No modelo que organiza o primeiro e o último core, na Figura 8b nota-se uma repetição de padrões: os erros se acumulam em áreas com um número reduzido de *cores*, enquanto configurações que permitem maior paralelismo (ou seja, mais *cores*) apresentam erros mais baixos. Assim como nas experiências anteriores, a estrutura determinística do conjunto de dados propicia uma convergência inicial mais fácil, mas não representa com precisão o comportamento do algoritmo em zonas onde o *overhead* de comunicação e o grau de queda da escalabilidade influenciam os tempos de execução.

Por último, na Figura 8c, o modelo linear apresenta erros consideráveis nas áreas com um número elevado de *cores* e entradas reduzidas, um padrão que lembra o que foi observado na *Matrix Multiplication*. O modelo mostra uma rápida convergência para configurações com entradas pequenas, mas tem dificuldades em se adaptar a comportamentos não lineares, especialmente em regiões com alto paralelismo, onde a interação complexa entre os fatores (*cores* e entradas) passa a ser mais relevante.

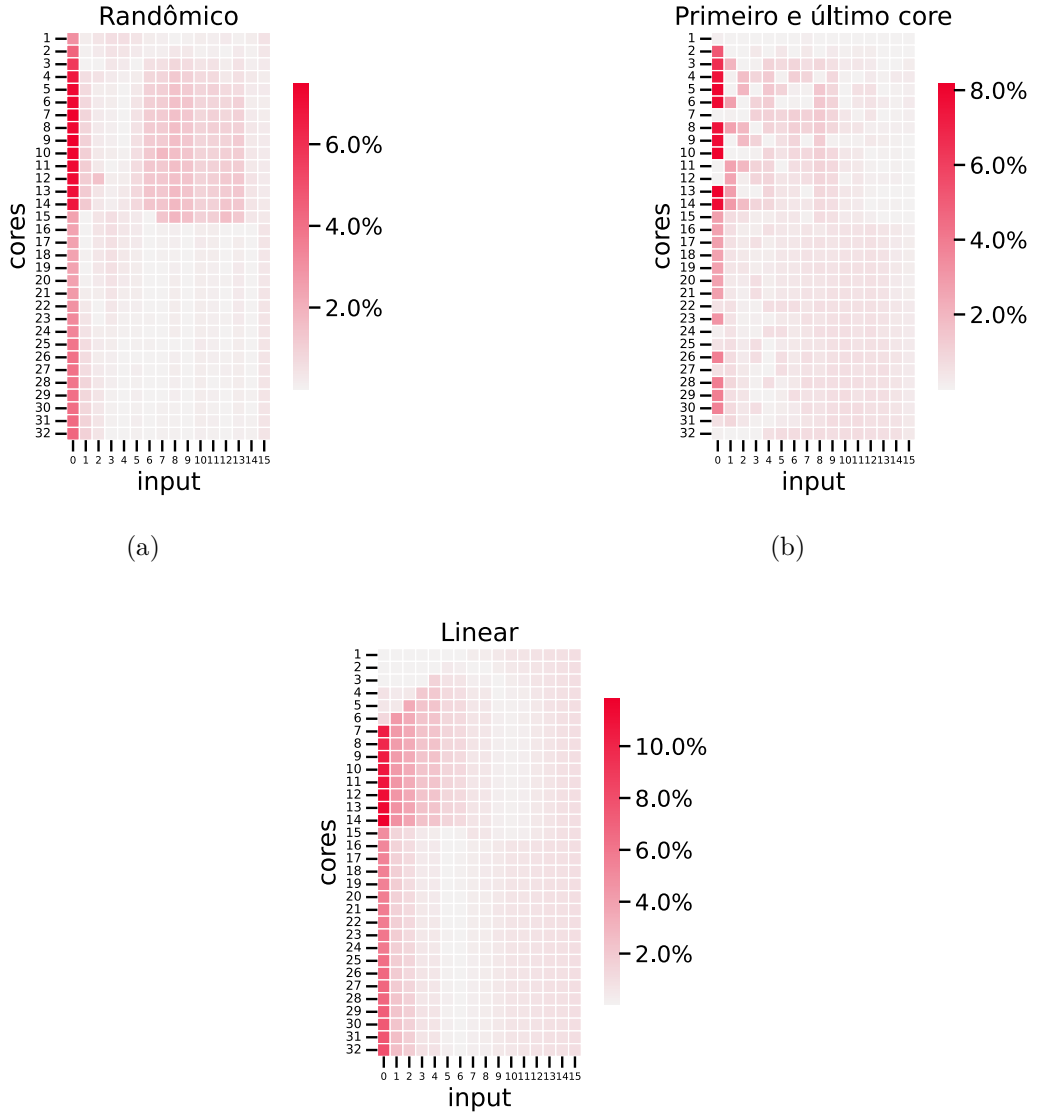

Figura 8 – Comparação do resíduo relativo absoluto entre a medição real e a estimativa do modelo *PascalRegressionModel* para diferentes estratégias de pré-processamento no algoritmo *Raytrace*. **(a)** Estratégia randômica. **(b)** Estratégia primeiro e último *core*. **(c)** Estratégia linear.

Dessa forma, os problemas destacados nos gráficos aparecem de maneira constante nos diferentes algoritmos examinados, sugerindo que as restrições podem estar ligadas tanto ao próprio modelo, que se baseia apenas nas *features* de *core* e *input*, quanto à organização do *dataset*, que afeta o aprendizado dos dados. Essas constatações apontam para a necessidade de desenvolver modelos mais robustos e adaptáveis, que consigam identificar os padrões não lineares existentes nos tempos de execução das aplicações paralelas em várias configurações.

A Figura 9 ilustra os ganhos e as perdas de tempo obtidos em cada aplicação

ao usar o *Pascal Regression Model*, em comparação com as medições reais. O algoritmo *BFS* demonstrou melhorias expressivas, com o pré-processamento linear resultando numa diminuição de 98,8% no tempo de execução, enquanto o pré-processamento randômico e primeiro e último *core* propiciou uma redução de 88,2% e 60,5%, respectivamente. Em contraste, o *Matrix Multiplication* teve um desempenho variado: o pré-processamento linear registrou uma melhoria de 96,0%, e o aleatório reduziu o tempo em 84,3%, mas o pré-processamento de primeiro e último *core* aumentou o tempo de execução em 33,0%. Por último, o *Raytrace* apresentou uma diminuição de 99,8% com o pré-processamento linear, além de uma redução de 73,9% no primeiro e último *core*. Por outro lado, o pré-processamento aleatório conseguiu reduzir o tempo em 96,3%. Dessa maneira, o método sugerido demonstra que a ferramenta tem grande potencial para otimizar as análises de escalabilidade.

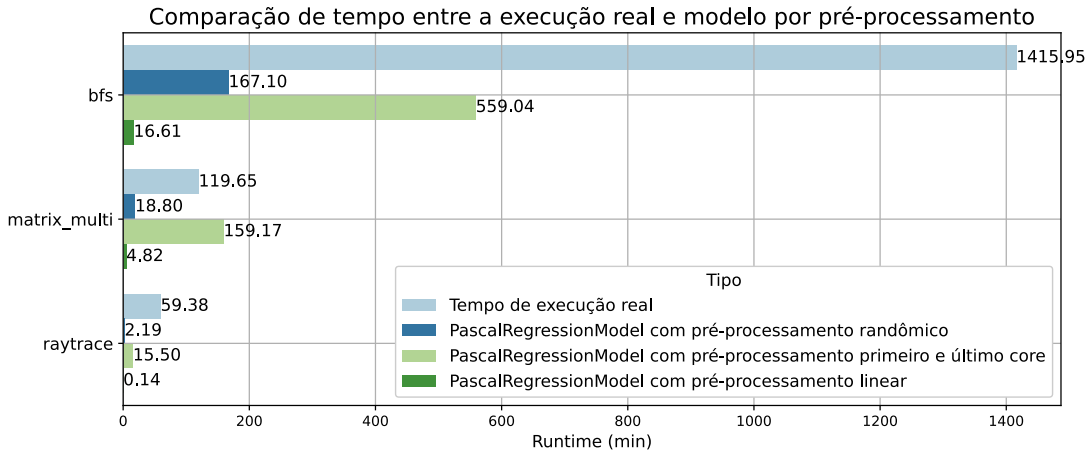

Figura 9 – Comparação do tempo total de execução entre a medição real e as estimativas do modelo para cada algoritmo e pré-processamento

Fonte: Autoria própria

5 CONCLUSÃO

Este trabalho demonstrou que a combinação de um modelo baseado em rede neural e a estratégia usada para calcular os tempos de execução de algoritmos paralelos é promissora, alcançando um equilíbrio aceitável entre precisão e tempo de análise reduzido. Os resultados ressaltam o potencial de tais modelos para simplificar o processo de análise da escalabilidade de algoritmos paralelos, contribuindo assim para um gerenciamento mais eficiente dos recursos computacionais.

No entanto, há espaço para melhorias na área de pré-processamento dos dados, para garantir um melhor equilíbrio e atenuar o risco de ajuste excessivo. Técnicas de ordenação na etapa de pré-processamento, como a Sequência de Halton (WANG; HICKERNELL, 2000), poderiam contribuir para um treinamento mais balanceado e com baixa discrepância. Além disso, os modelos atuais apresentam desafios ao lidar com cenários específicos que envolvem variabilidade de dados e casos extremos, o que pode afetar a generalização geral dos resultados.

Em conclusão, o modelo de pré-processamento linear oferece os maiores benefícios em termos de tempo, especialmente nos algoritmos *BFS* e *Matrix Multiplication*. Contudo, ele apresenta dificuldades em termos de generalização. O modelo que considera o primeiro e o último *core* mostra resultados variáveis e um desempenho insatisfatório na *Matrix Multiplication*, o que resulta em um aumento no tempo de execução da análise, contrariando o propósito do trabalho. Por outro lado, o modelo randômico continua sendo a abordagem mais sólida, proporcionando reduções significativas de tempo (entre 72% e 96%) ao mesmo tempo que mantém maior confiabilidade e capacidade de generalização. Considerando a necessidade de realizar previsões rápidas sem comprometer a precisão, o pré-processamento randômico permanece como a opção mais apropriada.

Por fim, para trabalhos futuros deve-se concentrar no refinamento do pipeline de pré-processamento para tratar dessas limitações e explorar arquiteturas alternativas ou modelos híbridos que possam oferecer maior robustez e adaptabilidade. Outrossim, adição de novas *features* no modelo para facilitar a captação de *overheads* de comunicação e sincronismo entre os núcleos de processamento, como as memórias cache L1, L2 e L3, a frequência operacional da CPU, a frequência da memória RAM, entre outros aspectos de recursos computacionais. Dessa forma, o desempenho do modelo pode ser otimizado ainda mais, melhorando sua aplicabilidade na análise de algoritmos paralelos do mundo real.

REFERÊNCIAS

ABRANTES, R. L. d. A. *Aplicações de computação paralela em otimização contínua*. 2008. Dissertação de Mestrado — Universidade de São Paulo, São Paulo, Brasil. Disponível em: https://www.teses.usp.br/teses/disponiveis/45/45134/tde-09072008-175128/publico/dissertacao.pdf.

AKIOKA, S.; MURAOKA, Y. Performance analysis of high performance computing applications on the cloud. In: *Proceedings of the 24th IEEE International Conference on Advanced Information Networking and Applications*. [S.l.]: IEEE, 2010. p. 1029–1036. Disponível em: https://doi.org/10.1109/AINA.2010.187.

AMDAHL, G. M. Validity of the single processor approach to achieving large scale computing capabilities. In: *Proceedings of the April 18-20, 1967, spring joint computer conference*. [S.l.: s.n.], 1967. p. 483–485.

ATTIG, N.; GIBBON, P.; LIPPERT, T. Trends in supercomputing: The european path to exascale. *Computer Physics Communications*, Elsevier, v. 182, n. 9, p. 2041–2046, 2011.

BARNEY, B. *Introduction to Parallel Computing*. 2021. Online; accessed 2024-11-12. Disponível em: https://computing.llnl.gov/tutorials/parallel_comp/.

BUYYA, R. et al. Cloud computing and emerging it platforms: Vision, hype, and reality for delivering computing as the 5th utility. *Future Generation Computer Systems*, Elsevier, v. 25, n. 6, p. 599–616, 2009.

CURTIS-MAURY, M. et al. Identifying energy-efficient concurrency levels using machine learning. In: *2007 IEEE International Conference on Cluster Computing*. [S.l.: s.n.], 2007. p. 488–495.

DONGARRA, J. et al. Accelerating numerical dense linear algebra calculations with gpus. *Numerical Computations with GPUs*, Springer, p. 1–26, 2019. Disponível em: https://doi.org/10.1007/978-3-030-14232-4_1.

FOSTER, I. *Designing and building parallel programs: concepts and tools for parallel software engineering*. [S.l.]: Addison-Wesley Longman Publishing Co., Inc., 1995.

GOIS, L. Supercomputador da ufrn tem avanço no poder computacional. *Portal da UFRN*, 2024.

GURNEY, K. *An Introduction to Neural Networks*. London: UCL Press Limited, 1997.

IPEK, E. et al. An approach to performance prediction for parallel applications. In: *European Conference on Parallel Processing*. [S.l.: s.n.], 2005. Disponível em: https://api.semanticscholar.org/CorpusID:15476966.

LAPPS. *PASCALSuite Analyzer*. 2025. https://gitlab.com/lappsufrn/pascalsuite-analyzer. Último acesso em: 19 de janeiro de 2025.

MOORE, G. Moore's law. *Electronics Magazine*, v. 38, n. 8, p. 114, 1965.

NISHTALA, R. et al. Scaling communication-intensive applications on bluegene/p using one-sided communication and overlap. In: *Parallel Distributed Processing, 2009. IPDPS 2009. IEEE International Symposium on*. [S.l.]: IEEE, 2009. p. 1–12.

PACHECO, P. *An Introduction to Parallel Programming*. San Francisco: Morgan Kaufmann, 2011.

PATTERSON, D. A.; HENNESSY, J. L. *Computer organization and design ARM edition: the hardware software interface*. [S.l.]: Morgan kaufmann, 2016.

RUSSEL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. Boston: Pearson Education, Inc., 2010.

SILVA, V. R. G. da et al. A minimally intrusive approach for automatic assessment of parallel performance scalability of shared-memory hpc applications. *Electronics*, v. 11, n. 5, 2022. ISSN 2079-9292. Disponível em: https://www.mdpi.com/2079-9292/11/5/689.

SINGH, K. et al. Predicting parallel application performance via machine learning approaches. *Concurrency and Computation: Practice and Experience*, v. 19, 2007. Disponível em: https://api.semanticscholar.org/CorpusID:264883473.

STERGIOU, C. et al. Secure integration of iot and cloud computing. *Future Generation Computer Systems*, Elsevier, v. 78, p. 964–975, 2018. Disponível em: https://doi.org/10.1016/j.future.2016.11.031.

WANG, S. C. Artificial neural network. In: *Interdisciplinary Computing in Java Programming*. Boston, MA: Springer, 2003, (The Springer International Series in Engineering and Computer Science, v. 743). p. 81–90.

WANG, X.; HICKERNELL, F. Randomized halton sequences. *Mathematical and Computer Modelling*, v. 32, n. 7, p. 887–899, 2000. ISSN 0895-7177. Disponível em: https://www.sciencedirect.com/science/article/pii/S0895717700001783.